# Database System
# [ECEG-4172]

## Chapter Five:

## Structured Query Language (SQL)

# Outline

- Introduction
- Simple Query Constructs and Syntax
- Nested Subqueries and Complex Queries
- Views
- Schema Definition in SQL
- Embedded and Dynamic SQL

# Introduction

- **Structured Query Language (SQL)** is a query language that is standardized for RDBMS.
- SQL-1 Standardized by ANSI in 1986
- SQL-2 Standardized in 1992
- SQL-3 standardization still ongoing
- SQL Statements (commonly referred to as 'queries') are run to ***retrieve*** or ***modify*** the requested information from the database
- SQL supports:
  - Data Definition Language (DDL), and
  - Data Manipulation Language (DML)

# SQL Data Definition Language (DDL)

- Part of the SQL language that permits database tables and constraints to be created, modified or deleted.
  - **CREATE TABLE** - creates a new database table.
  - **ALTER TABLE** - alters (changes) a database table.
  - **DROP TABLE** - deletes a database table.
  - **CREATE INDEX** - creates an index (search key).
  - **DROP INDEX** - deletes an index.
- DDL statements are used for schema definition of a relational database.

# SQL Data Manipulation Language (DML)

- It is part of the SQL syntax for executing queries to ***insert***, ***retrieve***, ***update***, or ***delete*** records.
  - **INSERT INTO** - inserts new data into a database table.
  - **SELECT** - extracts data from a database table.
  - **UPDATE** - updates data in a database table.
  - **DELETE** - deletes data from a database table.
- The four most common commands are also known as **SQL CRUD** statements.

# Review of
# Schema Definition in SQL

# Schema Creation and Modification

- SQL statement that is used to create a new database and the corresponding files for storing the database:

  CREATE DATABASE <database_name>

- **Schema** is a concept that is used to group database objects such as tables, constraints, views and permissions in the same application.
- The syntax for the command is:

  CREATE SCHEMA <schema_name> AUTHORIZATION <owner>

# Example

- Creating the database

Format: - ***CREATE DATABASE*** *database_name*

**CREATE DATABASE** [HHOffFurn]

**CREATE DATABASE** youtube

**CREATE DATABASE** My_First_Database

- Delete a database

Format: - ***DROP DATABASE*** *database_name*

**DROP DATABASE** My_First_Database

# Table Creation and Modification

- The SQL statement that is used to specify a new relation in a database:

**CREATE TABLE** <table_name> (

<column_name> <data_type> {column_constraint},

:

<column_name> <data_type> {column_constraint}

)

# Data Types

| Data Type | Description |
|---|---|
| **integer(size)** | Hold integers only.<br>The maximum number of digits are specified in parenthesis. |
| **int(size)** | |
| **smallint(size)** | |
| **tinyint(size)** | |
| **decimal(size,d)** | Hold numbers with fractions.<br>The maximum number of digits are specified in "size".<br>The maximum number of digits to the right of the decimal is specified in "d". |
| **numeric(size,d)** | |
| **char(size)** | Holds a fixed length string (can contain letters, numbers, and special characters).<br>The fixed size is specified in parenthesis. |
| **varchar(size)** | Holds a variable length string (can contain letters, numbers, and special characters).<br>The maximum size is specified in parenthesis. |
| **date(yyyymmdd)** | Holds a date |

# Example

Creating the tables

```
CREATE TABLE [Emplyees] (
    [empId] smallint IDENTITY (1, 1) NOT NULL,
    [name] varchar (30) NOT NULL,
    [sex] char (1) NOT NULL,
    [bDate] datetime NULL,
    [address] varchar (50) NULL,
    [empDate] datetime NULL,
    [position] varchar (20) NULL,
    [salary] float NULL
)
```

# …Example:

```
CREATE TABLE users (
    id int,
    username varchar (30),
    password varchar(20),
    PRIMARY KEY(id)
)
```

# …Example:

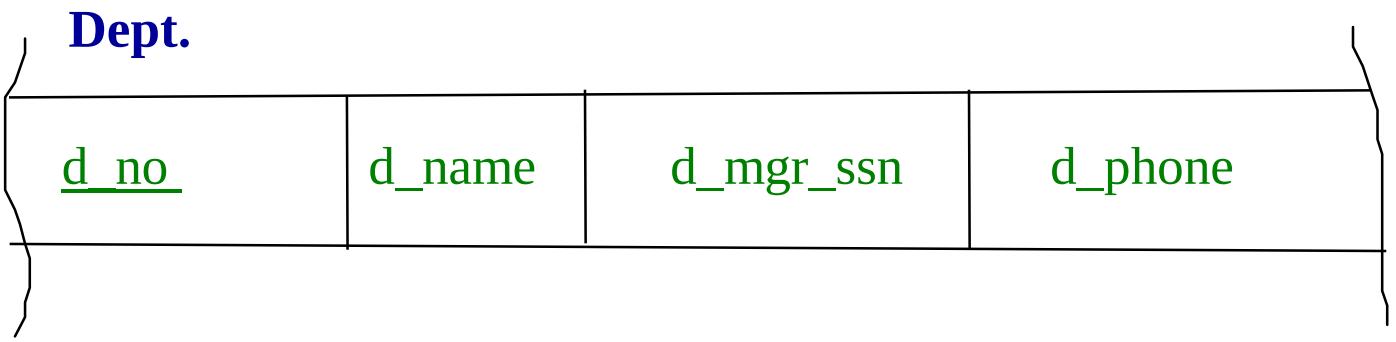
```
CREATE TABLE Person(
    LastName varchar,
    FirstName varchar,
    Address varchar,
    Age int
)
```

# ...Example:

```
CREATE TABLE employee (
    first varchar(15),
    last varchar(20),
    age number(3),
    address varchar(30),
    city varchar(20),
    state varchar(20)
     );
```

# ...Example:

Creates a table (schema)




```
CREATE TABLE  DEPT. (   d_no          Integer,
                        d_name        VarChar(15),
                        d_mgr_SSN     Char(9),
                        d_phone       Char(12)
        );
```

# NOT NULL & AUTO_INCREMENT

- Modifiers may be applied to attributes.
  - Two common modifiers are
    - **NOT NULL**:-data can't be added without this attribute having a value &
    - **DEFUALT**:-sets the data to the value that follows when no data is supplied.
- **AUTO_INCREMENT** modifier automatically writes a unique number into an attribute when no data is supplied.

# ...Example:

```
CREATE TABLE users (
    id int  NOT NULL AUTO_INCREMENT,
    username varchar (30) NOT NULL,
    password varchar(20) NOT NULL,
    PRIMARY KEY(id)
)
```

# ... Table Creation and Modification

- The primary key constraint in a relation is enforced by using the key word PRIMARY KEY.
- The referential integrity constraint in a relational database is implemented by the use of FOREIGN KEY.
  - Referential trigged actions:
    - ON DELETE {CASCADE | NO ACTION | SET DEFAULT | SET NULL}
    - ON UPDATE {CASCADE | NO ACTION | SET DEFAULT | SET NULL }

# . . . Table Creation and Modification

- The **ALTER TABLE** command allows modification (adding, changing, or dropping) of a column or constraint in a table.

```
ALTER TABLE <table_name>
    [ALTER COLUMN <column_name> <new_data_type>] |
    [ADD <column_definition> | <constraint>] |
    [DROP <column_name> | < constraint>]
```

# Cont'd...

- Add columns in an existing table.
  - *ALTER TABLE table_name ADD column_name datatype*

  Example: ALTER TABLE Person ADD Sex char(6)
- Delete columns in an existing table.
  - *ALTER TABLE table_name DROP column_name*

  Example: ALTER TABLE Person DROP Sex
- Delete a table.
  - *DROP TABLE table_name*

  Example: DROP TABLE Person

# Examples:

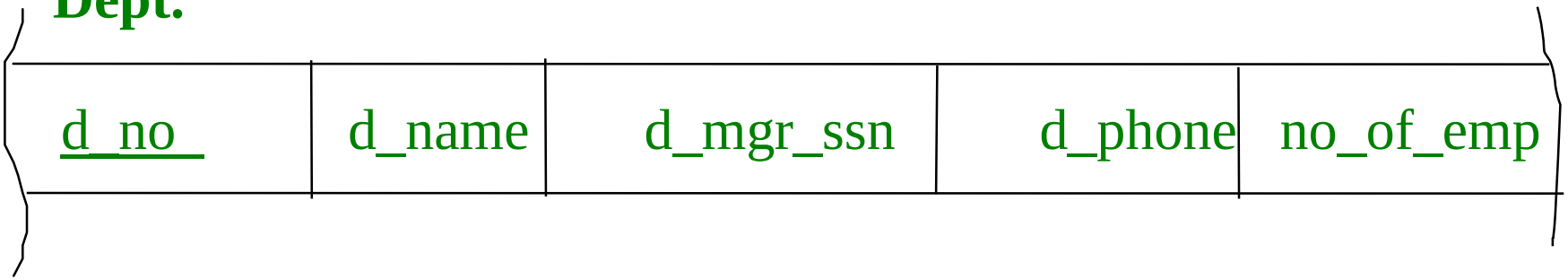
optional

- ALTER TABLE bacon ADD COLUMN samplecolumn varchar(10);
- ALTER TABLE bacon DROP COLUMN samplecolumn;
- DROP TABLE bacon;
- RENAME TABLE customers TO users;

# ...Example:

Alter table adds/drops *attributes*; Drop table drops the entire *table*.

**Dept.**




ALTER TABLE *DEPT*. ADD *no_of_emp* Integer;

ALTER TABLE *DEPT*. DROP *no_of_emp*;

DROP TABLE *Dept.;*

# Constraints

- Constraints are the rules enforced on data columns on table.
  - These are used to limit the type of data that can go into a table.
  - This ensures the accuracy and reliability of the data in the database.
- Constraints could be *column level* or *table level*.
  - Column level constraints are applied only to one column, whereas table level constraints are applied to the whole table.

# Cont'd...

- **NOT NULL** Constraint: Ensures that a column cannot have NULL value.
- **DEFAULT** Constraint: Provides a default value for a column when none is specified.
- **UNIQUE** Constraint: Ensures that all values in a column are different.
- **PRIMARY Key**: Uniquely identified each rows/records in a database table.
- **FOREIGN Key**: Uniquely identified a rows/records in any another database table.
- **CHECK Constraint**: The CHECK constraint ensures that all values in a column satisfy certain conditions.
- **INDEX**: Use to create and retrieve data from the database very quickly.

# Cont'd...

- Constraints can be specified when a table is created with the CREATE TABLE statement or you can use ALTER TABLE statement to create constraints even after the table is created.

# Example:

- CREATE TABLE CUSTOMERS(

```
ID   INT              NOT NULL,
NAME VARCHAR (20)     NOT NULL,
AGE  INT              NOT NULL,
ADDRESS  CHAR (25) ,
SALARY   DECIMAL (18, 2),
PRIMARY KEY (ID)
)
```

- ALTER TABLE CUSTOMERS

```
MODIFY SALARY  DECIMAL (18, 2) NOT NULL;
```

# Adding constraints

- Default Constraint

```
ALTER TABLE [Emplyees] ADD
    CONSTRAINT [DF_Emplyees_salary]
DEFAULT
    (
            0
    ) FOR [salary]
```

- Unique Constraint

```
ALTER TABLE [Resources] ADD
    CONSTRAINT [UQ_Resources_stockNo] UNIQUE NONCLUSTERED
    (
            [stockNo], [regDate]
    )
```

# Example:

- CREATE TABLE CUSTOMERS(

```
ID   INT              NOT NULL,
NAME VARCHAR (20)     NOT NULL,
AGE  INT              NOT NULL,
ADDRESS  CHAR (25) ,
SALARY   DECIMAL (18, 2) DEFAULT 5000.00,
PRIMARY KEY (ID)
        )
```

- ALTER TABLE CUSTOMERS

```
MODIFY SALARY  DECIMAL (18, 2) DEFAULT 5000.00;
```

- ALTER TABLE CUSTOMERS

```
ALTER COLUMN SALARY DROP DEFAULT;
```

# Example:

- CREATE TABLE CUSTOMERS(
  ID INT NOT NULL,
  NAME VARCHAR (20) NOT NULL,
  AGE INT NOT NULL UNIQUE,
  ADDRESS CHAR (25) ,
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID)
  );
- ALTER TABLE CUSTOMERS
  MODIFY AGE INT NOT NULL UNIQUE;
- ALTER TABLE CUSTOMERS
  ADD CONSTRAINT myUniqueConstraint UNIQUE(AGE, SALARY);
- ALTER TABLE CUSTOMERS
  DROP CONSTRAINT myUniqueConstraint;

# ... Example

- Adding constraints
  - Key Constraint

```
ALTER TABLE [Emplyees] WITH NOCHECK ADD
  CONSTRAINT [PK_Emplyees] PRIMARY KEY CLUSTERED
  (
        [empId]
  )
```

  - Foreign Key Constraint (Relationship)

```
ALTER TABLE [Receipts] ADD
  CONSTRAINT [FK_Receipts_Employees] FOREIGN KEY
  (
        [empId]
  ) REFERENCES [Employees] (
        [empId]
  )
```

# Example:

- CREATE TABLE CUSTOMERS(
  ID INT NOT NULL,
  NAME VARCHAR (20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25) ,
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID)

);

- ALTER TABLE CUSTOMER ADD PRIMARY KEY (ID);
- ALTER TABLE CUSTOMERS DROP PRIMARY KEY ;

# Example:

- CREATE TABLE CUSTOMERS(
  ID INT NOT NULL,
  NAME VARCHAR (20) NOT NULL,
  AGE INT NOT NULL,
  ADDRESS CHAR (25) ,
  SALARY DECIMAL (18, 2),
  PRIMARY KEY (ID, NAME)

);

- ALTER TABLE CUSTOMERS
  ADD CONSTRAINT PK_CUSTID PRIMARY KEY (ID, NAME);
- ALTER TABLE CUSTOMERS DROP PRIMARY KEY ;

# Simple Query Constructs and Syntax

# SELECT-FROM-WHERE Statement

- The **SELECT-FROM-WHERE** Statement is **data reading query**
  - Simplest Syntax

    SELECT <column_list>
    FROM <table_list>
    WHERE <condition>
  - The **DISTINCT** key word on the SELECT clause:

    SELECT DISTINCT <column_list>
  - The **LIKE** key word for test of string pattern:

    S LIKE P
    S NOT LIKE P

# Example:

- Select
  - All Data

```
SELECT [empId], [name], [sex], [bDate], [address], [empDate],
   [position], [salary]
FROM [Emplyees]
```

  - Single Data

```
SELECT [empId], [name], [sex], [bDate], [address], [empDate],
   [position], [salary]
FROM [Emplyees]
WHERE [empId] = 1
```

# ...Examples:

- Retrieve the **surname** and **firstname** of the **customers** table
  - SELECT surname, firstname
    FROM customers;
- Retrieve all the data from the **customers** table
  - SELECT * FROM customers;
- Retrieve one example of each different customer **surname** in the **customers** table
  - SELECT DISTINCT surname
    FROM customers;

# ...Examples:

- Retrieve the salary of every employee salary values
  - SELECT ALL salary
    FROM employee ;
- Retrieve all distinct salary values of employees
  - SELECT DISTINCT salary
    FROM employee ;
- Retrieve the **name** and **state** of all customers whose **state** is ‘CA’ from the **customers** table
  - SELECT name, state
    FROM customers
    WHERE state=‘CA’;

# ...Examples:

- Retrieve the birthdate and address of the employee(s) whose name is 'John B. Smith'.
  - SELECT bDate, address
    FROM employee
    WHERE fName='John' AND mInit='B' AND
    lName='Smith';
- Retrieve the **id** and **name** of all customers whose **id** less than or equal to **54** from the **customers** table
  - SELECT id, name
    FROM customers
    WHERE id<=54;

# Use of BETWEEN, NOT BETWEEN, IN, NOT IN

Example:

```
SELECT id, name
FROM customers
WHERE id BETWEEN 25 AND 30;
```

- The condition **(id BETWEEN 25 AND 30)** in example above is equivalent to the condition **((id>= 25) AND (id<= 30)).**
- **id IN(68,6,40)** is equivalent to **((id= 68) OR (id= 6) OR (id= 40)).**

# Use of IS NULL, IS NOT NULL

- You can check if a column contains NULL values by IS NULL or IS NOT NULL.

For example,

```
SELECT * FROM products WHERE product Code IS NULL;
```

# Substring Pattern Matching (using LIKE and NOT LIKE)

- The **LIKE** key word is used for test of string pattern in conjunction with two reserved characters
  - **Underscore(_)**: matches a single character
  - **The percentage sign(%)**:-matches zero or more characters.

  Example:
  - '%frogs', 'frogs%', '%frogs%', '_ smith%', '_ _ _ 5 _ _'etc

# Examples:

Select firstname and lastname of all records in which the last name starts with ‘Bank’

– SELECT firstname, lastname

FROM users

WHERE lastname LIKE ‘Bank%’

Select the name for every record whose email address is not of the form ‘something@authors.com’

– SELECT firstname, lastname

FROM users

WHERE email NOT LIKE ‘%@authors.com’

# Additional Examples:

- SELECT name
  FROM items
  WHERE name LIKE ‘new%’
- SELECT city
  FROM customers
  WHERE city LIKE ‘h%d’
- SELECT name
  FROM items
  WHERE name LIKE ‘_ boxes of frogs’

# Alias

- An **alias** is an alternate name for a column or an expression.
- Aliases are typically used for displaying output in a user-friendly manner.
- They also serve as shorthand when referring to columns or expressions to reduce typing.
- SQL offers a formalized way of inserting aliases. The **AS** keyword is inserted between the column or expression and the alias.

# Cont'd...

- Column name alias

  Format:

  **SELECT** column_name AS column_alias

  **FROM** table_name

  **Example**

  SELECT LastName AS Family, FirstName AS Name

  FROM Persons

# Cont'd...

- Table name alias

  Format:

  **SELECT** table_alias.column_name

  **FROM** table_name **AS** table_alias

**Example**

```
SELECT LastName, FirstName
FROM Persons AS Employees
```

```
SELECT e.lName AS employee_name, s.lName AS   supervisor_name
FROM employee AS e, employee AS s
WHERE e.superSsn=s.ssn;
```

# INSERT, UPDATE and DELETE

- The **INSERT** statement
  - Syntax

    INSERT INTO <table_name>| <view_name> [(column_list)]
    data_values

# Cont'd...

- The **UPDATE** statement
  - Syntax

```
UPDATE <table_name>
SET <column_name> = <value> [,..n]
WHERE <condtion>
```

- The **DELETE** statement
  - Syntax

```
DELETE FROM <table_name>| <view_name>
WHERE <condtion>
```

# Cont'd...

Examples:

- Insert

```
INSERT INTO [HHOffFurn].[dbo].[Emplyees]([name], [sex], [bDate], [address],
    [salary])
VALUES('AAA', 'M', '02-02-1978', ' ', 750)
```

- Update

```
UPDATE [Emplyees]
SET [name]='BBB', [empDate]='1-1-2001'
WHERE [empID] = 1
```

- Delete

```
DELETE FROM [Emplyees]
WHERE [empID] = 1
```

# Cont'd...

**Examples:**

```
INSERT INTO items
VALUES('101', 'bacon strips', '9.95', '1', '0');

INSERT INTO items (id, name, cost, seller_id, bids)
VALUES('102', 'fish n chips', '7.99', '1', '0');

INSERT INTO items (id, cost, name)
VALUES('103',  '7.77', 'beef on stick ');
```

Optional when all values ar
listed and they are in order
as the definition of the tabl

Not Optional

# Cont'd...

Examples:

```
UPDATE items
SET name='frog paste', bids=66
WHERE id='106';

DELETE FROM items
WHERE id='104';
```

# Schema Definition in SQL Continued

# Index Creation and Modification

- **Indexes** are the heart of fast data access that provides fast data access.
- An index for a table is managed by an external table which consists of the search key (index attribute) and a pointer to the location of the data as columns.

## Syntax

**CREATE** [ UNIQUE ] INDEX <index name>
**ON** <table name> ( <column name> [ <order> ] { , <column name> [ <order> ] } )
[ CLUSTER ] **;**

- The keywords UNIQUE and CLUSTER are optional.
  - For a clustered index the data is both stored and sorted on the index key.

# Cont'd...

Example:

```
CREATE  NONCLUSTERED  INDEX empName
        ON Employees (name  ASC )
```

- The DROP command is also used with indexes to drop an existing database in a table.
- The syntax  for the command is:

```
DROP INDEX <index_name> [,...n ]
```

Example:

```
DROP INDEX empName
```

# Nested Subqueries and Complex Queries

# Subqueries

- Subqueries can be used in different ways:
  - In the WHERE clause to form nested queries,
  - In set operations such as UNION, EXCEPT, …, and
  - In the FROM clause as constant tables

# Nested Queries

- **SQL SELECT** statements can be contained in the WHERE clause of another SQL statement to form ***Nested queries*****.**
- The SELECT statement that contains the nested query is said to be the ***outer query***.
- Subqueries in a nested SQL statement can produce scalar value (constant) or table.
- TEST Operators in subquery:
  - IN
  - ALL
  - ANY
  - EXISTS

# Cont'd...

Example:

```
SELECT Name, Address
    FROM Customers AS c
    WHERE EXISTS (SELECT *
    FROM Projects AS p
    WHERE c.CustId=p.CustId)
```

# SET Operation of Queries

- The set operations union, intersection and difference (UNION, INTERSECT and EXCEPT) can be used in SQL statements that will consist of two subqueries as the operations are binary.
- Syntax

  **(Subquery 1)**
  **UNION | INTERSECT | EXCEPT |**
  **(Subquery 2)**

- The set operators do not include duplicate in the resulting table.
  - If all the resulting rows are to be retrieved the ALL operator is used with the set operators to prevent duplicate elimination.(i.e UNION ALL)

# Cont'd...

Example:

```
SELECT name, cost, bids
FROM items
WHERE bids>190
      UNION
  SELECT name, cost, bids
  FROM items
  WHERE bids>1000
```

- **Note**: Each of the queries must produce the same number of columns.

# Joined Tables

- Reading data from more than one table with the use of joined tables in the FROM clause.
  - ***Cross Product***

    <left_table> CROSS JOIN <right_table>
  - ***Natural Join***

    <left_table> NATURAL JOIN <right_table>
  - ***Theta Join***

    <left_table> JOIN <right_table> ON<condition>
  - ***Outer Join***

    <left_table> [RIGHT | LEFT | FULL] OUTER {NATURAL} JOIN <right_table> {ON<condition>}

# Cont'd...

Examples:

- To retrieve the name and address of every employee who works for the 'Research' department, it may be easier first to specify the join of the EMPLOYEE and DEPARTMENT relations, and then to select the desired tuples and attributes.

```
SELECT fName, lName, address
FROM (employee JOIN department ON dNo=dNumber)
WHERE dName='Research';
```

# Cont'd...

Examples:

```
SELECT customers.name, items.name
FROM customers LEFT OUTER JOIN items
        ON customers.id=seller_id;
```

- LEFT…implies… take all of outputs of **customers** table.
- If LEFT is changed by RIGHT, it will take all outputs of **items** table.

# Ordering Query Results

- To specify the order in which the resulting table is organized, the **ORDER BY** clause is used in the statement.

Syntax:

```
SELECT <column_list>
  FROM <table_list>
  WHERE <condition>
  ORDER BY {<order_column> [ASC | DESC]}[,..n]
```

Note:

- **ASC** (ascend) is a alphabetical and numerical order (**optional**)
- **DESC** (descend) is a reverse alphabetical and numerical order

# Cont'd...

## Examples:

- SELECT *

  FROM Persons

  ORDER BY LastName

- SELECT FirstName, LastName

  FROM Persons

  ORDER BY LastName DESC

- SELECT Company, OrderNumber

  FROM Orders

  ORDER BY Company DESC, OrderNumber ASC

# Aggregate Function in SQL

- SQL allows grouping of resulting rows in a query so that aggregate functions can be applied to make analysis and summary.
- The aggregate functions supported by the SQL statement are:
  - **Summation:** Returns the total sum of a column
    **SUM** (<column_name>)
  - **Average:** Returns the average value of a column
    **AVG** (<column_name>)
  - **Minimum:** Returns the lowest value of a column
    **MIN** (<column_name>)
  - **Maximum:** Returns the highest value of a column
    **MAX** (<column_name>)
  - **Count:** Returns the number of rows (without a NULL value) of a column
    **COUNT** (<column_name> | *)

# Cont'd...

**Examples:**

- Find the sum of the salaries of all employees, the maximum salary, the minimum salary, and the average salary.
  - SELECT SUM (salary), MAX (salary), MIN(salary), AVG (salary)
    FROM employee;
- Retrieve the total number of employees in the company
  - SELECT COUNT(*)
    FROM employee;

# Cont'd...

**Examples:**

- Retrieve the total number of employees in the 'Research' department
  - SELECT COUNT(*)
    FROM employee, department
    WHERE dNo=dNumber AND dName='Research';

# Grouping in SQL Statement

- The **GROUP BY** clause is used after the **WHERE** clause to group the result of the SQL statement.

Syntax:

```
GROUP BY <group_column>[,..n]
HAVING <condition>
```

- The columns that can be included in the HAVING clause are:
  - Aggregated column from the FROM clause tables, or
  - Unaggregated columns from the GROUP BY list

# Cont'd...

**Examples:**

- For each department, retrieve the department number, the number of employees in the department, and their average salary.
  - SELECT dNo, COUNT (*), AVG (salary)
    FROM employee
    GROUP BY dNo;
- A query that counts the number of teams a given Project is having

```
SELECT p.Name, COUNT(t.Name) AS NoOfTeams
FROM Projects p JOIN Teams t ON p.PrjId=t.PrjId
GROUP BY p.Name
```

# Cont'd...

## Examples:

- To add on a condition for grouping such as only projects that are having teams more than 2.

```
SELECT p.Name, COUNT(t.Name) AS NoOfTeams
FROM Projects p JOIN Teams t ON p.PrjId=t.PrjId
GROUP BY p.Name
HAVING COUNT(t.Name)>2
```

# Cont'd...

## Examples:

- For only the projects that are having a name starting with 'A', the query can be written as:

```
SELECT p.Name, COUNT(t.Name) AS NoOfTeams
FROM Projects p JOIN Teams t ON p.PrjId=t.PrjId
GROUP BY p.Name
HAVING p.Name LIKE 'A%'
```

# Summary of SQL Statement

- In summary the general form the SELECT statement is as follows

```
SELECT select_list
FROM table_source
[WHERE search_condition]
[GROUP BY group_by_expression]
[HAVING search_condition]
[ORDER BY order_expression [ASC | DESC] ]
```

# Views

- **View** in the context of SQL is a virtual table that is derived from one or more tables in an alternative way.

Syntax

```
CREATE VIEW <view_name>
AS <select_statement>
```

- Views as the database objects can also be dropped using the DROP statement as follows:

```
DROP VIEW <view_name>
```

# Cont'd...

## Examples:

- To create table from already created table:
  - SELECT id, name, bids

    FROM items

    ORDER BY bids DESC LIMIT 10;
  - This will select the **top ten** based on **bids.** We should **always run** this query to obtain this table.
- To create **view** from already created table:
  - CREATE VIEW mostbids AS

    SELECT id, name, bids

    FROM items

    ORDER BY bids DESC LIMIT 10;

# Cont'd...

## Examples:

- We can use the created table for selecting purpose
  - SELECT name, bids
    FROM mostbids;
- To delete/drop **view**:
  - DROP VIEW mostbids;

# Embedded and Dynamic SQL

- The SQL standard defines embeddings of SQL in a variety of programming languages, such as C, Cobol, Pascal, Java, PL/I, and Fortran.
  - A language in which SQL queries are embedded is referred to as **a host language**, and the SQL structures permitted in the host language constitute **embedded SQL**.

# Cont'd...

- The **dynamic SQL** component of SQL allows programs to construct and submit SQL queries **at run time**.
- In contrast, embedded SQL statements must be completely present at compile time; they are compiled by the embedded SQL preprocessor.
- Using dynamic SQL, programs can create SQL queries as strings at run time (perhaps based on input from the user) and can either have them executed immediately or have them prepared for subsequent use. Preparing a dynamic SQL statement compiles it, and subsequent uses of the prepared statement use the compiled version.

# End